

HerbHacks — AI-Based Herbal Skin Care Solution

Project Report | March 2026

1. Executive Summary

HerbHacks is a full-stack AI-powered web application that helps users understand their skin health and discover herbal remedies. The system uses a custom-trained deep learning model to classify skin types from photos, scans cosmetic product labels for harmful chemicals via OCR, and recommends herbal treatments. It was built for the **CVMU Hackathon 4.0**.

2. Architecture Overview

```
graph TD
    A[React Frontend<br/>Vite + Tailwind] -->|HTTP / Axios| B[FastAPI Backend<br/>Python 3]
    B --> C[MobileNetV2 Model<br/>skin_model.h5]
    B --> D[Tesseract OCR<br/>Ingredient Scanner]
    B --> E[Supabase Cloud<br/>Auth + DB + Storage]
    B --> F[SQLite Local DB<br/>herbhacks.db]
    A --> E
```

3. Technology Stack

Backend

Component	Technology
Web Framework	FastAPI (Python)
ML Framework	TensorFlow 2.15.0
Base Model	MobileNetV2 (Transfer Learning)
OCR Engine	Tesseract + pytesseract + OpenCV
Auth (local)	SQLAlchemy + bcrypt + JWT (HS256)
Auth (cloud)	Supabase JWT
Database (local)	SQLite (herbhacks.db)
Database (cloud)	Supabase PostgreSQL + Storage
Server	Uvicorn on 0.0.0.0:5000

Frontend

Component	Technology
Framework	React 19 + Vite 7

Styling	Tailwind CSS v4
Routing	React Router DOM v7
HTTP Client	Axios
Maps	Leaflet + React-Leaflet
Face Detection	MediaPipe Tasks Vision
Charts	Recharts
Cloud Client	@supabase/supabase-js v2
Icons	Lucide React

4. Machine Learning Model

Architecture

- **Base Model:** MobileNetV2 (pre-trained on ImageNet, weights frozen)
- **Custom Head:**
 - GlobalAveragePooling2D
 - Dense(128, relu)
 - Dropout(0.3)
 - Dense(3, softmax) — final output layer
- **Model file:** backend/skin_model.h5 (~11.5 MB)

Training Configuration

Parameter	Value
Input Size	224 × 224 × 3
Batch Size	32
Max Epochs	15
Optimizer	Adam
Loss	Categorical Cross-Entropy
Early Stopping	patience=3 (monitors val_loss)
Best Checkpoint	Saves on best val_accuracy

Data Augmentation

- Rotation ±20°, Width/Height shift ±20%, Shear ±20%, Zoom ±20%
- Horizontal flip, nearest fill mode

Output Classes

Class	Description
-------	-------------

acne	Acne-prone skin
normal	Balanced/healthy skin
oily	Oily/sebaceous skin

Skin Health Score Formula

```
health_score = (normal_prob × 100) + (oily_prob × 40) + (acne_prob × 20)
                Clamped between 10 - 100
```

5. API Endpoints

Method	Endpoint	Auth	Description
GET	/	None	Health check
POST	/auth/register	None	Register user (local DB)
POST	/auth/login	None	Login → JWT token
GET	/auth/me	Bearer JWT	Get current user
POST	/predict	Optional	Upload image file → skin analysis
POST	/predict-frame	Optional	Base64 webcam frame → skin analysis
POST	/analyze	Supabase JWT	Analyze from URL (Supabase)
POST	/scan-ingredients	None	Upload product label → OCR analysis
GET	/harmful-chemicals	None	List all harmful chemicals DB

6. Core Features

 AI Skin Analysis (Scan Page)

- Upload a photo **or** use live webcam (MediaPipe face detection)
- Runs through MobileNetV2 classifier
- Returns: skin type, confidence %, class probabilities, skin health score
- Provides herbal **remedies**, **description**, and **care tips** from `remedies.json`
- **Safeguard:** OCR pre-check on uploaded image — if product label text is detected (2+ trigger words), rejects image and redirects to Ingredient Scanner

 Ingredient Scanner

- Upload a cosmetic product label photo
- OpenCV pre-processing → Tesseract OCR → extract raw text
- Cross-references against `harmful_chemicals.json` database
- Returns list of harmful chemicals detected with warnings

 Dashboard

- User scan history (fetched from Supabase `scan_history` table)

- Charts and analytics via Recharts

Nearby Stores

- Uses browser Geolocation API
- Interactive map (Leaflet) showing nearby herbal / ayurvedic / organic stores
- Store details and "Get Directions" integration

Authentication

- **Dual auth system:**
 - Local: SQLAlchemy + SQLite (register/login with bcrypt hashed passwords, HS256 JWT)
 - Cloud: Supabase JWT for protected `/analyze` endpoint
- `ProtectedRoute` component guards all sensitive pages

Cloud Persistence (Supabase)

- Scan images uploaded to **Supabase Storage** bucket (`scan-images`)
- Scan results saved to `scan_history` table (user_id, image_url, disease_name, confidence_score, remedies)
- Runs as a **background task** so it never blocks the API response

7. Project File Structure

```
herbal/
├── backend/
│   ├── main.py           ← FastAPI app + all API routes
│   ├── train_model.py    ← MobileNetV2 training script
│   ├── skin_model.h5     ← Trained model weights (~11.5 MB)
│   ├── requirements.txt  ← Python dependencies
│   ├── herbhacks.db      ← SQLite local user database
│   ├── auth/
│   │   ├── routes.py     ← /auth/register, /auth/login, /auth/me
│   │   ├── models.py     ← SQLAlchemy User model
│   │   ├── schemas.py    ← Pydantic schemas
│   │   ├── utils.py      ← Hashing, JWT helpers
│   │   └── database.py   ← SQLAlchemy engine/session
│   ├── model/
│   │   └── predict.py     ← Inference pipeline (load, cache, predict)
│   ├── ocr/
│   │   └── ingredient_scanner.py ← OpenCV + Tesseract OCR pipeline
│   └── data/
│       ├── remedies.json ← Herbal remedies per skin type
│       └── harmful_chemicals.json ← Harmful chemicals database
├── frontend/
│   ├── src/
│   │   ├── App.jsx       ← Router + AuthProvider wrapper
│   │   └── pages/
│   │       ├── Home.jsx  ← Landing page
│   │       ├── Auth.jsx  ← Login / Register
│   │       ├── Scan.jsx  ← AI Skin Analysis (main feature)
│   │       ├── IngredientScanner.jsx ← OCR product scanner
│   │       ├── Dashboard.jsx ← User history & analytics
│   │       └── NearbyStores.jsx ← Map + store finder
```

```
| | | components/      ← Navbar, ProtectedRoute, etc.
| | | context/         ← AuthContext
| | |   └─ i18n/        ← Internationalization
| | └─ package.json
| └─ data/
|   └─ train/           ← Training images (acne, normal, oily)
|     └─ val/           ← Validation images
| └─ CVMU_HACKATHON_4.0PPT.pptx ← Hackathon presentation
```

8. Running the Project

Backend

```
cd backend
# Install dependencies
pip install -r requirements.txt
# Start server (port 5000)
python main.py
```

Frontend

```
cd frontend
npm install
npm run dev # Starts on http://localhost:5173
```

Train Model (optional)

```
cd backend
python train_model.py
```

9. Known Limitations & Improvement Areas

Area	Current State	Suggested Improvement
Skin Classes	Only 3 (acne, normal, oily)	Expand to 10+ disease classes
Auth	Dual system (local + Supabase) can be fragmented	Unify to single Supabase auth
Model Accuracy	MobileNetV2 with small dataset	Fine-tune with larger dermatology dataset
OCR Accuracy	Tesseract performs poorly on curved/low-res labels	Add image super-resolution pre-processing
CORS	Hardcoded to localhost:5173/5174	Move to environment variables
JWT Secret	Hardcoded in source code	Move to .env

Production Build	No deployment config present	Add Docker + CI/CD pipeline
---------------------	------------------------------	-----------------------------

10. Summary

HerbHacks successfully demonstrates a working end-to-end AI-powered herbal skin care assistant with:

-  Real-time skin type classification using transfer learning
-  Herbal remedy recommendations
-  OCR-based harmful ingredient detection
-  Supabase-backed scan history and image storage
-  Interactive nearby store map
-  Secure JWT authentication
-  Modern React SPA with glassmorphism UI

Built for **CVMU Hackathon 4.0** — March 2026